10-Day Interview Plan

QA Automation Engineer | Big MNC Preparation

Language: **JavaScript** (Playwright with Node.js). DSA and API automation are also done in JavaScript, so you practise one language all day.

Daily Timetable

Time	Topic	Learn	Practice
8:00-11:00	Playwright Automation	1 hr	1 hr practice + 1 hr real-time scenario / mini project
11:00-12:00	Break, lunch		
12:00-2:00	DSA	1 hr	1 hr (solve without looking at solutions)
2:00-3:00	Break		
3:00-4:00	Git/GitHub	30 min	30 min
4:00-6:00	API Testing (Swagger/Postman)	1 hr	1 hr
7:00-8:00	SQL (PostgreSQL)	30 min	30 min
8:30-9:30	Daily revision (free time)	Review notes	5 interview questions

Git and SQL slots are 1 hour each, so they are split 30/30. The 3-hour Playwright slot includes an extra hour for real-time scenarios. Day 10 is for mock interviews.

10-Day Progress Tracker

Day	Playwright	DSA	Git	API	SQL
Day 1	[ ]	[ ]	[ ]	[ ]	[ ]
Day 2	[ ]	[ ]	[ ]	[ ]	[ ]
Day 3	[ ]	[ ]	[ ]	[ ]	[ ]
Day 4	[ ]	[ ]	[ ]	[ ]	[ ]
Day 5	[ ]	[ ]	[ ]	[ ]	[ ]
Day 6	[ ]	[ ]	[ ]	[ ]	[ ]
Day 7	[ ]	[ ]	[ ]	[ ]	[ ]
Day 8	[ ]	[ ]	[ ]	[ ]	[ ]
Day 9	[ ]	[ ]	[ ]	[ ]	[ ]
Day 10	[ ]	[ ]	[ ]	[ ]	[ ]

1. Playwright Automation (8:00-11:00)

Day	New topic	Practice and real-time scenario
1	Node.js and VS Code setup, *npm init playwright@latest*, playwright.config.js, async/await basics, first test, codegen	Automate login on saucedemo.com (valid and invalid login)
2	Locators (getByRole, getByText, getByLabel, CSS, XPath), actions, auto-wait	Fill the signup form on automationexercise.com
3	Web-first assertions (expect), dropdowns, checkboxes, dialogs (alerts), iframes (frameLocator), multiple tabs	Handle a popup, a new-tab link and an iframe on the-internet.herokuapp.com
4	Playwright Test runner: test.describe, hooks (beforeEach, beforeAll), tags, fixtures, data-driven testing (JSON/CSV, loops)	Run login with 5 data sets: valid, invalid and empty cases
5	Page Object Model with JS classes, custom fixtures, .env/dotenv, utilities	Convert Day 1-4 scripts into POM (LoginPage, HomePage, CartPage)
6	Reporting (HTML reporter, Allure), screenshots and video on failure, trace viewer, retries, test.step	Make a test fail on purpose and debug it with the trace viewer
7	File upload/download, network interception (page.route), mocking, storageState (reuse login)	Mock an API response to test an "empty cart" UI; skip repeated logins
8	Parallel execution (workers, fullyParallel), projects for cross-browser, headless mode, CI with GitHub Actions	Run the suite on Chromium, Firefox and WebKit in parallel
9	Mini-project: end-to-end flow (login, search, add to cart, checkout)	Build it on saucedemo or automationexercise with full POM and reports
10	Revision: framework walkthrough, flaky tests, interview Q&A;	Prepare a 3-minute explanation of your framework

Interview questions to cover: Playwright vs Selenium/Cypress, auto-wait, async/await and Promises, locator strategy, handling flaky tests, handling authentication (storageState), POM benefits, parallel execution, CI integration.

2. DSA (12:00-2:00)

Day	New topic	Practice problems
1	JavaScript basics (let/const, loops, functions), String basics	Reverse a string, palindrome, count vowels, count words
2	Strings in depth (split, join, slice, replace, charAt)	Anagram check, first non-repeating character, remove duplicates, character frequency
3	Arrays 1	Largest and second largest, reverse, missing number, remove duplicates (sorted)
4	Arrays 2	Two Sum, move zeros, merge sorted arrays, rotate array, Kadane's algorithm
5	Array methods (map, filter, reduce, find, sort, slice, splice, forEach)	Remove duplicates from an array, sort an array of objects, flatten an array, common elements
6	Map, Set and Object as a hash map	Word count, Two Sum using a Map, group anagrams, first repeating element
7	Functions, recursion, closures, classes (OOP in practice)	Factorial, Fibonacci, prime, GCD, Armstrong number, a closure counter
8	Two pointers, sliding window, binary search, sorting	Valid palindrome, max sum of subarray of size k, binary search, bubble sort
9	ES6+ (destructuring, spread, arrow functions), higher-order functions, interview questions	var vs let vs const, == vs ===, hoisting, Map vs Object, Promises vs async/await
10	Timed mock coding round	Solve 5 mixed problems in 90 minutes, then explain your approach aloud

Use LeetCode (Easy), HackerRank and GeeksforGeeks. Aim for **4-6 problems a day**.

3. Git/GitHub (3:00-4:00)

Day	New topic	Practice
1	init, clone, add, commit, status, log, diff	Create a repo locally and make 5 commits
2	Branching: branch, switch, merge	Create a feature branch and merge it into main
3	GitHub: remote, push, pull, fetch, PAT/SSH	Push your repo to GitHub, clone it to another folder, pull changes
4	Pull requests, merge conflicts	Create a conflict on purpose and resolve it
5	stash, reset, revert, restore, commit --amend	Undo a wrong commit in three different ways
6	rebase, cherry-pick, tags, .gitignore	Cherry-pick one commit from a feature branch to main
7	Team workflows: GitFlow, code review, forks	Fork a repo, make a change, raise a PR
8	GitHub Actions basics	Run your Playwright tests automatically on every push
9	Push your full framework with a README, plus Jenkins basics	A clean, interview-ready repo link for your resume
10	Interview Q&A;	merge vs rebase, reset vs revert, fetch vs pull, how you resolve conflicts

4. API Testing: Swagger/Postman (4:00-6:00)

Day	New topic	Practice and real-time scenario
1	HTTP, REST, methods, status codes, headers, JSON	Send GET requests to jsonplaceholder.typicode.com
2	Postman: collections, params, environments and variables	Create a collection with 5 requests and a "dev" environment
3	POST, PUT, PATCH, DELETE, auth (Basic, Bearer), body types	Full CRUD on restful-booker.herokuapp.com
4	Postman test scripts (pm.test), chaining requests, pre-request scripts	Pass the token or booking ID from one request to the next
5	Collection Runner, data-driven runs (CSV/JSON), Newman	Run 10 data rows through the Create Booking API
6	Swagger/OpenAPI: read a spec, Swagger UI, import into Postman	Test petstore.swagger.io: add, get, update and delete a pet
7	API testing in Playwright: request fixture, APIRequestContext, GET/POST with expect	Rewrite your Postman GET/POST tests as Playwright API tests
8	Playwright API tests: auth headers, request chaining, response validation, combining API setup with UI tests	Create a booking via API, then verify it in the UI; validate response fields
9	Negative testing, schema validation, E2E API flow	Test invalid token (401), missing field (400), wrong ID (404); write an E2E API test
10	Interview Q&A;	PUT vs PATCH, 401 vs 403, idempotency, status codes, testing an API without documentation

5. SQL, PostgreSQL (7:00-8:00)

Day	New topic	Practice
1	Install PostgreSQL and pgAdmin, CREATE TABLE, INSERT, SELECT, WHERE	Create *employees* and *departments* tables with 10 rows each
2	ORDER BY, LIMIT, DISTINCT, LIKE, IN, BETWEEN, NULL	10 filtering queries
3	Aggregates, GROUP BY, HAVING	Employees per department, average salary per department
4	Joins (INNER, LEFT, RIGHT, FULL)	Employees with department names; employees with no department
5	Subqueries, EXISTS, IN	Employees earning above the average salary
6	UPDATE, DELETE, constraints (PK, FK, UNIQUE), ALTER	DELETE vs TRUNCATE vs DROP with examples
7	Window functions (ROW_NUMBER, RANK, DENSE_RANK), CTE	Nth highest salary, top 2 earners per department
8	Classic interview queries	Find duplicates, delete duplicates, second highest salary, self join
9	SQL in QA testing: data validation, DB check after API call, indexes, views, transactions	Create a booking through the API, then verify it in the DB
10	Mock interview	Solve 10 queries in 30 minutes

Day 10: Final Checklist

- Run a mock interview with a friend, or answer questions aloud and record yourself.
- Prepare your **self-introduction** and a **project explanation** (framework, tools, your role).
- Revise manual testing basics: SDLC/STLC, test case vs scenario, bug life cycle, severity vs priority, smoke vs sanity vs regression, Agile/Scrum.
- Make sure your resume lists Playwright, JavaScript, Postman, SQL, Git and CI, with a GitHub link.

Daily Rules

- Do not copy and paste solutions. Type them yourself.
- Keep a notebook of **errors you hit and how you fixed them**; interviewers love debugging stories.
- If you fall behind, drop extra practice rather than skipping a topic.